claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_095035a0-e9f\agent-a8d1d5200fa75b258.jsonl`
- SHA-256: `31b254e7823730634dd5d702f1716673f2d5a075df463342503f4725f9c7d933`
- Source modified: `2026-06-13T19:16:41+00:00`
- Imported at: `2026-07-08T16:00:00+00:00`
- project: `wf_095035a0-e9f`
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`

Transcript

1. user (2026-06-13T19:15:42.420Z)

Adversarially VERIFY this fusion-P2 review finding by reading the actual code. Try to REFUTE it; only confirm real=true if it genuinely holds against the code. Default to real=false if uncertain or if it's a nitpick/false-positive.

FINDING: {"title":"The tracknet substrate branch of _build_runner is never exercised end-to-end","severity":"low","file":"backend/pipeline/segmenters/fusion_hybrid.py:58-68 ; tests/test_fusion_hybrid.py:47","detail":"_build_runner has a real branch on fusion.substrate ('tracknet' builds TrackNetRunner with queue_length + fusion_substrate='tracknet' params; default builds WasbRunner). Every fusion-hybrid test patches _build_runner out entirely (patch.object(FusionSegmenter, '_build_runner', return_value=(_runner(), {...}))), so neither branch runs and the runner_params dict ('fusion_substrate' tag, queue_length) is never asserted. test_config only checks that substrate='tracknet' parses into config, not that the segmenter routes on it. This is partially a GPU concern (constructing the real runners), but the routing logic + the returned params dict are pure and could be unit-tested by calling _build_runner directly with TrackNetConfig/WasbConfig mocked.","fix":"Add a direct unit test of _build_runner: call seg._build_runner({'fusion':{'substrate':'tracknet'}}) and seg._build_runner({}) with TrackNetRunner/WasbRunner patched, asserting the right runner class is constructed and runner_params carries fusion_substrate plus weights_path (and queue_length for tracknet)."}

Repo root = C:/Users/avidu/Projects/badminton-highlight-indexer. The change implements fusion P2 (MULTI_SIGNAL_FUSION_PLAN.md): a new default-OFF `FusionSegmenter` (backend/pipeline/segmenters/fusion_hybrid.py) that subclasses TrajectoryHybridSegmenter, persists shuttle net_crossings (Gate A) + optional person-cue features into candidate stats, and optionally gates the AI handoff. Pure feature math is in backend/pipeline/detectors/fusion_features.py. NEW files: backend/pipeline/detectors/fusion_features.py, backend/pipeline/segmenters/fusion_hybrid.py. EDITED files (use `git --no-pager diff -- <path>` to see edits): backend/pipeline/segmenters/trajectory_hybrid.py, backend/pipeline/detectors/person_detector.py, backend/config/models.py, backend/pipeline/segmenters/__init__.py. READ the actual files. The work is uncommitted in the working tree on branch master. Quote file:line. Only report REAL issues; rank by severity (high/med/low).

2. assistant (2026-06-13T19:15:45.235Z)

I'll verify this finding by reading the actual code. Let me start by examining the relevant files.

3. user (2026-06-13T19:15:46.193Z)

```
1       """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3       Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4       TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5       do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
```

```
6       shuttle-seeded window via two overridable seams the engine exposes:
7
8       - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9         GPU-free, computed from the trajectory we already have); and, only when the person cue
10        is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11        (`fusion_features`), persisting them into ``candidate_segments.stats`` for the learned
12        fusion (P3). The person path lazily builds ONE `PersonDetector` ? so the default path
13        never imports torch / touches the GPU.
14      - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
15        (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
16        dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
17        **Persisted candidates remain the full superset** regardless, so the offline
experiment
18        harness can re-score any variant.
19
20      Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
21      nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
22      and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
23      segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
24      experiment harness (EXPERIMENT_HARNESS.md), never silently.
25      """
26      from typing import Any, Dict, List, Optional, Tuple
27
28      from backend.pipeline.segmenters.base import segmenter_registry
29      from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
30      from backend.pipeline.detectors.base import DetectorRunner
31      from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
32      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
33      from backend.pipeline.detectors import rally_gate
34      from backend.pipeline.detectors import fusion_features as ff
35
36
37      class FusionSegmenter(TrajectoryHybridSegmenter):
38          """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""
39
40          family = "fusion"
41          display_label = "Fusion"
42
43          def __init__(self, db: Any, config: Any):
44              super().__init__(db, config)
45              # Built lazily only if the person cue is enabled (keeps torch/GPU off the
default path).
46              self._person: Optional[Any] = None
47
48          @property
49          def name(self) -> str:
50              return "fusion_hybrid"
51
52          @property
53          def description(self) -> str:
54              return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
55                      "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
56                      "AI provider sets semantic boundaries.")
57
58          def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
```

```
            substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
            if substrate == "tracknet":
                cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
                return TrackNetRunner(cfg), {
                    "weights_path": cfg.weights_path,
                    "queue_length": cfg.queue_length,
                    "fusion_substrate": "tracknet",
                }
            wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
            return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}

        def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
                                    frame_width: float, video_path: str,
                                    idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
            stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
            # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
            # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
            # over the whole trajectory by rally_gate (camera-agnostic).
            gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
min_crossings=0)
            stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0

            fcfg = idx_cfg.get("fusion", {})
            if fcfg.get("cue_flags", {}).get("person", False):
                stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
            return stats

        def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
                             frame_width: float, video_path: str,
                             fcfg: Dict[str, Any]) -> Dict[str, float]:
            """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
            frame range and compute the scale/translation-invariant person features. Lazily
builds
            ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
            # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
            from backend.pipeline.detectors.person_detector import PersonDetector
            if self._person is None:
                self._person = PersonDetector(self.config)

            frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
            start_f, end_f = int(cw["start"] * fps), int(cw["end"] * fps)
            det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
            fw = det["frame_width"] or frame_width
            fh = det["frame_height"] or 0.0

            poly = fcfg.get("court_polygon")
            polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
            net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
            shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
            return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
                                              court_polygon=polygon, net=net)

        def _select_for_handoff(self, windows: List[Dict[str, Any]],
                                window_stats: List[Dict[str, Any]],
                                idx_cfg: Dict[str, Any]) -> List[int]:
            """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
```

```
113             (0) keep every window ? identical to the substrate. Persisted candidates are
114             untouched (full superset)."""
115             fcfg = idx_cfg.get("fusion", {})
116             min_crossings = int(fcfg.get("min_crossings", 0))
117             min_players = int(fcfg.get("min_players", 0))
118             person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
119
120             keep: List[int] = []
121             for i, st in enumerate(window_stats):
122                 if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
123                     continue
124                 if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
125                     continue
126                 keep.append(i)
127             return keep
128
129
130     segmenter_registry.register(FusionSegmenter)
131
```

4. user (2026-06-13T19:15:46.353Z)

```
1       """FusionSegmenter tests (mocked runner/provider/db ? no GPU/WSL).
2
3       Proves: net_crossings is persisted; the person cue is OFF by default (no person-feature
4       keys, PersonDetector never built); the persisted candidate set is the full superset of
the
5       substrate windows; served Gate A drops sub-threshold windows from the AI handoff while
6       persistence keeps the superset; and the person cue, when enabled, adds the person
features.
7       """
8       from types import SimpleNamespace
9       from unittest.mock import MagicMock, patch
10
11      from backend.pipeline.detectors.fusion_features import PERSON_FEATURE_NAMES
12      from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
13
14      # Window A [1,4]s: shuttle alternates across the net every frame -> many crossings (a
rally).
15      # Window B [10,12]s: shuttle sits at the net level (held) -> 0 crossings. fps=30.
16      WIN_A = {"start": 1.0, "end": 4.0, "active_frame_count": 90, "peak_velocity": 0.4,
17               "mean_velocity": 0.2, "track_id_count": 2, "chunk_index": 0}
18      WIN_B = {"start": 10.0, "end": 12.0, "active_frame_count": 60, "peak_velocity": 0.1,
19               "mean_velocity": 0.05, "track_id_count": 1, "chunk_index": 0}
20
21
22      def _points():
23          pts = []
24          for f in range(30, 120):       # window A frames: y flips 100<->900 around the net
(~500)
25              pts.append(SimpleNamespace(frame=f, visible=True, x=500.0, y=100.0 if f % 2 == 0
else 900.0))
26          for f in range(300, 360):      # window B frames: y == net level -> deadband -> no
crossings
27              pts.append(SimpleNamespace(frame=f, visible=True, x=500.0, y=500.0))
28          return pts
29
30
31      def _runner():
32          r = MagicMock()
33          r.healthcheck.return_value = (True, "env ok")
34          r.run_predict.return_value = "out.csv"
35          r.parse_trajectory_csv.return_value = _points()
36          r.trajectory_to_action_windows.return_value = [dict(WIN_A), dict(WIN_B)]
```

```
        return r


    def _run(indexing=None, provider_segments=None):
        provider = MagicMock()
        provider.analyze_video.return_value = (provider_segments or [{"start_time": 0.5,
"end_time": 2.0}], {})
        db = MagicMock()
        db.add_candidate_segment.side_effect = [101, 102, 103, 104]
        db.add_play_segment.return_value = 200

        with patch.object(FusionSegmenter, "_build_runner", return_value=(_runner(),
{"weights_path": "w"})), \
             patch.object(FusionSegmenter, "_probe_fps_width", return_value=(30.0, 1280.0)),
\
             patch("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
return_value=30.0), \
             patch("backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk",
return_value=True), \
             patch("backend.pipeline.segmenters.trajectory_hybrid.provider_registry") as
preg, \
             patch("os.environ.get", return_value="dummy_key"):
            preg.get.return_value = provider
            seg = FusionSegmenter(db, {"indexing": indexing or {}})
            res = seg.process_video("v1", "clip.mp4")
            return db, provider, res


    def _stats_calls(db):
        """Persisted stats dicts, in call order."""
        return [kw["stats"] for _a, kw in db.add_candidate_segment.call_args_list]


    def test_net_crossings_persisted_and_person_off_by_default():
        db, _provider, _res = _run()
        stats = _stats_calls(db)
        assert len(stats) == 2                              # both windows persisted
(superset)
        assert all("net_crossings" in s for s in stats)
        # Window A (rally) crosses many times; window B (held) zero.
        assert stats[0]["net_crossings"] >= 2
        assert stats[1]["net_crossings"] == 0.0
        # Person cue OFF by default -> NONE of the person features present.
        assert all(not any(k in s for k in PERSON_FEATURE_NAMES) for s in stats)
        # The 4 base motion keys are still there.
        assert all({"peak_velocity", "mean_velocity", "active_frame_count",
"track_id_count"} <= set(s)
                   for s in stats)


    def test_persisted_candidates_are_superset_of_substrate_windows():
        db, _provider, _res = _run()
        spans = [(kw["start_time"], kw["end_time"]) for _a, kw in
db.add_candidate_segment.call_args_list]
        assert spans == [(1.0, 4.0), (10.0, 12.0)]          # exactly the substrate windows,
unfiltered


    def test_default_hands_off_all_windows():
        _db, provider, _res = _run()                        # min_crossings default 0 -> no
gating
        assert provider.analyze_video.call_count == 2


    def test_gate_a_drops_low_crossing_window_from_handoff_only():
```

```
91          db, provider, _res = _run(indexing={"fusion": {"min_crossings": 2}})
92          # Persistence is still the full superset (offline substrate intact)...
93          assert len(_stats_calls(db)) == 2
94          # ...but only the high-crossing window A reaches the AI handoff.
95          assert provider.analyze_video.call_count == 1
96
97
98      def test_person_cue_adds_features_when_enabled():
99          fake_pd = MagicMock()
100         fake_pd.detections_per_frame.return_value = {
101             "frames": {30: [(1, (10, 10, 30, 40))], 31: [(1, (12, 10, 32, 40))]},
102             "fps": 30.0, "frame_width": 1000.0, "frame_height": 1000.0,
103         }
104         with patch("backend.pipeline.detectors.person_detector.PersonDetector",
return_value=fake_pd):
105             db, _provider, _res = _run(indexing={"fusion": {"cue_flags": {"person": True}}})
106         stats = _stats_calls(db)
107         assert all(all(k in s for k in PERSON_FEATURE_NAMES) for s in stats)
108         assert all("net_crossings" in s for s in stats)
109         fake_pd.detections_per_frame.assert_called()         # the person detector WAS used
110
```

5. assistant (2026-06-13T19:15:50.143Z)

Now let me verify the claims in the finding. Let me check the parent class to understand `_build_runner`, `from_indexing_cfg` signatures, and look at the config test mentioned.

6. user (2026-06-13T19:15:51.121Z)

```
1       """Shared engine for trajectory-substrate hybrid segmenters.
2
3       tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4       standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5       now the single home for the whole pipeline shape:
6
7           healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8           -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10      A concrete subclass supplies only its identity and runner wiring:
11      - ``family``        ? config section under ``indexing.*``, output subdir, and
12                            the ``<family>-hybrid-<model>`` detector tag in metadata.
13      - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14      - ``name`` / ``description`` properties (the registry contract).
15      - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16                            detection_params extras such as weights_path).
17
18      This base is deliberately NOT registered; only concrete subclasses register.
19      """
20      import os
21      import time
22      import logging
23      import concurrent.futures
24      from typing import Any, Dict, List, Optional, Tuple
25
26      import cv2
27
28      from backend.pipeline.segmenters.base import BasePlaySegmenter
29      from backend.utils.video import get_video_duration, extract_video_chunk
30      from backend.sports import sport_registry
31      from backend.providers import provider_registry
32      from backend.pipeline.detectors.base import DetectorRunner
33
34      logger = logging.getLogger(__name__)
35
36
```

```
    def _fmt_ts(seconds: float) -> str:
        seconds = max(0, int(seconds))
        h, rem = divmod(seconds, 3600)
        m, s = divmod(rem, 60)
        return f"{h:02d}:{m:02d}:{s:02d}"


    class TrajectoryHybridSegmenter(BasePlaySegmenter):
        """Trajectory-detector hybrid: precise candidate rallies + AI semantic
boundaries."""

        #: config section under `indexing` (velocity_threshold lives there), the
        #: output subdir for trajectories, and the metadata detector-tag prefix.
        family: str = ""
        #: human-readable label used in log lines.
        display_label: str = ""

        def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
            """Return (runner, detector-specific detection_params extras)."""
            raise NotImplementedError

        @staticmethod
        def _probe_fps_width(video_path: str) -> tuple:
            """Return (fps, frame_width) for a video, with sensible fallbacks."""
            cap = cv2.VideoCapture(video_path)
            fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
            width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
            cap.release()
            return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)

        @staticmethod
        def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
            """Provider-reported exchange count, else a duration-based estimate.

            One canonical implementation ? the twins had drifted (wasb relied on
            ``int(None)`` raising into the fallback; same result, by accident).
            """
            shot_count = segment.get("shuttle_exchange_count")
            if shot_count is None:
                return max(3, int(duration / 0.8))
            try:
                return int(shot_count)
            except (ValueError, TypeError):
                return max(3, int(duration / 0.8))

        # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
-------
        def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
                                    frame_width: float, video_path: str,
                                    idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
            """Stats dict persisted into ``candidate_segments.stats`` for one window. The
BASE
            returns exactly the 4 motion keys, so the trajectory twins persist byte-
identical
            rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
            features). ``points`` are the whole-video trajectory points
(TrajectoryPoint)."""
            return {
                "peak_velocity": cw["peak_velocity"],
                "mean_velocity": cw["mean_velocity"],
                "active_frame_count": cw["active_frame_count"],
                "track_id_count": cw["track_id_count"],
            }

```

```
96          def _select_for_handoff(self, windows: List[Dict[str, Any]],
97                                  window_stats: List[Dict[str, Any]],
98                                  idx_cfg: Dict[str, Any]) -> List[int]:
99              """Indices of the candidate windows that proceed to the AI handoff (and thus may
100             become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
101             ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
102             Persisted candidates are NOT affected ? they remain the full superset for
offline
103             re-scoring."""
104             return list(range(len(windows)))
105
106         def process_video(self, video_id: str, video_path: str,  # type: ignore[override]
107                           player_pool: Optional[List[str]] = None,
108                           max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
109             # override-ignore: the ABC declares the Result contract (Success|Failure)
110             # but every live segmenter still returns a bare list ? the documented
111             # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
112             label = self.display_label
113             # --- Sport profile + AI provider wiring (identical across segmenters) ---
114             sport_name = self.config.get("sport", "badminton")
115             try:
116                 sport_profile = sport_registry.get(sport_name)
117             except KeyError as e:
118                 logger.error(str(e))
119                 return []
120
121             idx_cfg = self.config.get("indexing", {})
122             provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
123             model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
124
125             api_key_env_var = f"{provider_name.upper()}_API_KEY"
126             api_key = os.environ.get(api_key_env_var)
127             if not api_key:
128                 logger.error(
129                     f"{api_key_env_var} environment variable is not set! "
130                     f"To run the {provider_name} handoff, set your API key. "
131                     f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
132                 )
133                 return []
134             try:
135                 provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
136             except KeyError as e:
137                 logger.error(str(e))
138                 return []
139
140             video_duration = get_video_duration(video_path)
141             if video_duration <= 0:
142                 logger.error("Invalid video duration.")
143                 return []
144             fps, frame_width = self._probe_fps_width(video_path)
145
146             # --- Runner config + windowing thresholds ---
147             runner, runner_params = self._build_runner(idx_cfg)
148
149             velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
150             merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
151             min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
152             handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
153             ai_max_workers = idx_cfg.get("ai_max_workers", 5)
154             log_candidates = idx_cfg.get("log_yolo_candidates", True)
155             store_candidates = idx_cfg.get("store_yolo_candidates", True)
156
```

```
            detection_params = {
                "detector": self.name,
                "velocity_thresh": velocity_thresh,
                "merge_gap": merge_gap,
                "min_action_window_duration": min_window_duration,
                **runner_params,
            }

            # --- Phase 1: env healthcheck (fail fast with a clear message) ---
            ok, msg = runner.healthcheck()
            if not ok:
                logger.error("%s WSL environment not ready: %s", label, msg)
                return []
            logger.info("%s WSL env OK: %s", label, msg)

            # --- Phase 2: trajectory -> candidate rally windows ---
            # Trajectory detectors process the whole video in one pass (they carry
            # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
            t_start = time.time()
            out_dir = os.path.join("output", self.family)
            csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
            if not csv_path:
                logger.error("%s produced no trajectory; aborting.", label)
                return []

            points = runner.parse_trajectory_csv(csv_path)
            logger.info("Parsed %d trajectory points (%d visible).",
                        len(points), sum(1 for p in points if p.visible))

            all_candidate_windows = runner.trajectory_to_action_windows(
                points, fps=fps, frame_width=frame_width, chunk_start=0.0,
                velocity_thresh=velocity_thresh, merge_gap=merge_gap,
                min_window_duration=min_window_duration, chunk_index=0,
            )
            logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
                        label, time.time() - t_start, len(all_candidate_windows))

            if log_candidates:
                for cw in all_candidate_windows:
                    logger.info(f"[{label} rally]
{_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
                                f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
                                f"peak_v={cw['peak_velocity']:.3f}
mean_v={cw['mean_velocity']:.3f}")

            # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
fusion
            # segmenter adds net_crossings + person-cue features. Computed once, reused for
both
            # persistence and the handoff gate below.
            window_stats = [
                self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
idx_cfg)
                for cw in all_candidate_windows
            ]

            # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
            candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
            if store_candidates:
                for i, cw in enumerate(all_candidate_windows):
                    candidate_ids[i] = self.db.add_candidate_segment(
                        video_id, detector=self.name,
                        start_time=cw["start"], end_time=cw["end"],
                        chunk_index=cw["chunk_index"], confidence=min(1.0,
```

```
cw["peak_velocity"]),
                        stats=window_stats[i], detection_params=detection_params,
                    )

            if not all_candidate_windows:
                return []

            # --- Phase 3: AI provider handoff over padded candidate clips ---
            # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted
            # candidates above stay the full superset ? only the served play_segments are
gated.
            handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
idx_cfg)
            handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
            os.makedirs(handoff_dir, exist_ok=True)
            logger.info("Phase 3: %s validation on %d candidate windows...",
                        provider_name, len(handoff_indices))

            def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
                w_start = max(0, window["start"] - handoff_padding)
                w_end = min(video_duration, window["end"] + handoff_padding)
                w_dur = w_end - w_start
                w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
                if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
                    return []
                prompt = sport_profile.get_prompt(chunk_duration=w_dur)
                segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
                                                     label=f"Handoff {wi+1}")
                valid = []
                for seg in segments:
                    try:
                        seg["start_time"] = float(seg["start_time"]) + w_start
                        seg["end_time"] = float(seg["end_time"]) + w_start
                        seg["_candidate_id"] = candidate_ids[wi]
                        valid.append(seg)
                    except (ValueError, TypeError, KeyError) as e:
                        logger.warning("Dropping segment with unparseable timestamps: %s ?
%s", seg, e)
                try:
                    os.remove(w_path)
                except OSError:
                    pass
                return valid

            ai_segments: List[dict] = []
            with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
                futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
                           for wi in handoff_indices]
                for future in futures:
                    ai_segments.extend(future.result())
            try:
                os.rmdir(handoff_dir)
            except OSError:
                pass
            logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))

            # --- Phase 4: DB population + candidate linking ---
            segments_data = []
            for segment in ai_segments:
                try:
```

```
272                     start = float(segment["start_time"])
273                     end = float(segment["end_time"])
274                     dur = end - start
275                     metadata = {
276                         "shot_count": self._shot_count_from(segment, dur),
277                         "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
278                         "detector": f"{self.family}-hybrid-{model_name}",
279                     }
280                     segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
281                     if segment_id:
282                         candidate_id = segment.get("_candidate_id")
283                         if candidate_id is not None:
284                             self.db.link_candidate_to_segment(candidate_id, segment_id)
285                         segments_data.append({
286                             "id": segment_id, "start_time": start, "end_time": end,
287                             "duration": dur, "metadata": metadata,
288                         })
289                 except Exception as e:
290                     logger.warning("Skipping invalid segment struct: %s", e)
291
292             return segments_data
293
```

7. user (2026-06-13T19:15:52.208Z)

```
docs\CODE_MAP.md:50:### 1B. WASB shuttle substrate (P2 of wasb_hybrid)
docs\CODE_MAP.md:114:- `@backend/pipeline/segmenters/wasb_hybrid.py::WasbHybridSegmenter` (name
`'wasb_hybrid'`, the LIVE shuttle substrate; MIT, ships pretrained badminton weights) ? thin
subclass: `family='wasb'`, wires `WasbRunner`/`WasbConfig`; `detection_params` extras =
`weights_path` only.
docs\CODE_MAP.md:140:- `@backend/eval/training.py` ? learned-segmenter data layer.
`::YOLO_FEATURE_NAMES` (5-d), `::extract_yolo_features` (? reads `row['stats']` dict from DB;
missing fields default 0.0), `::label_candidates` (same IoU `match_intervals` as evaluator),
`::build_training_set`->(X,y,intervals); `feature_fn=` is the substrate plug-in seam.
backend\config\models.py:55:    """Typed config for the live WASB shuttle substrate
(indexing.wasb).
backend\config\models.py:88:    # Which trajectory substrate seeds the windows (shuttle stays
primary).
backend\config\models.py:89:    substrate: Literal["wasb", "tracknet"] = "wasb"
backend\config\models.py:91:    # indexing.<family>.velocity_threshold). Default matches the
substrates' 0.02; set
docs\COMMERCIALIZATION.md:85:  cost, no external dep) and matches the Phase-4 roadmap (substrate
= WASB
backend\reporting\spec.yaml:94:  # for good results with the shuttle-tracking substrates.
backend\pipeline\segmenters\wasb_hybrid.py:5:badminton weights and runs today, so this is the
live shuttle-motion substrate.
tests\test_config.py:29:    """The fusion block defaults must be no-ops: wasb substrate, no
cues, no gating, no mask."""
tests\test_config.py:32:    assert f.substrate == "wasb"
tests\test_config.py:45:                "substrate": "tracknet", "min_crossings": 2,
tests\test_config.py:50:        assert f.substrate == "tracknet"
tests\test_config_wasb.py:5:shuttle substrate's config) was discarded ? config.json overrides
did nothing.
docs\EXPERIMENT_HARNESS.md:50:stored WASB trajectories. The candidate-segments table (`stats`
JSON) is the persistence substrate.
docs\EXPERIMENT_HARNESS.md:88:### 3.5 Offline re-scoring substrate (persist once, re-score
forever)
docs\archives\COMMERCIAL_READINESS_REVIEW.md:71:   - Validate real-video performance of the
current substrates (WASB is live default).
backend\pipeline\segmenters\trajectory_hybrid.py:1:"""Shared engine for trajectory-substrate
hybrid segmenters.
docs\archives\decisions\DECISIONS.md:153:**Status:** Proposed (substrate pending data)
docs\archives\decisions\DECISIONS.md:167:Three candidate substrates were weighed:
```

```
docs\archives\decisions\DECISIONS.md:183:2. **Choose the substrate from data, not assumption.**
A learned refiner can only sharpen
docs\archives\decisions\DECISIONS.md:199:### Update (2026-05-26): Phase 3 evidence ? substrate
is WASB, not YOLO
docs\archives\decisions\DECISIONS.md:218:shuttle signal), the offline segmenter's substrate is
**WASB shuttle-motion** (ADR-001),
docs\archives\decisions\DECISIONS.md:227:substrate-agnostic training-data + classifier layers
already built still apply: they will
docs\archives\decisions\DECISIONS.md:302:After building the WASB shuttle-trajectory substrate
(ADR-001), a Gemini-teacher ? local-student distillation
docs\GOLDEN_SET_PHASE1_VIDEOS.md:40:A controlled study: table-tennis ball detection was **22.5%
@ 30 fps ? 93.2% @ 60 fps** (120 fps ? 60 fps) ([Nature Sci
Rep](https://www.nature.com/articles/s41598-023-42966-6)); OpenTTGames records at 120 fps. The
ball/shuttle is the camera-invariant rally signal (our WASB substrate, ADR-001), so fps is
**not** a nicety for fast sports.
backend\eval\experiment.py:6:substrate) already exists ? this module composes it. **No new
scoring math lives
backend\pipeline\segmenters\fusion_hybrid.py:3:Shuttle stays PRIMARY: this reuses an existing
trajectory substrate (WASB by default,
backend\pipeline\segmenters\fusion_hybrid.py:20:Defaults reproduce the substrate exactly:
net_crossings/person features are persisted but
backend\pipeline\segmenters\fusion_hybrid.py:59:        substrate = idx_cfg.get("fusion",
{}).get("substrate", "wasb")
backend\pipeline\segmenters\fusion_hybrid.py:60:        if substrate == "tracknet":
backend\pipeline\segmenters\fusion_hybrid.py:65:                "fusion_substrate": "tracknet",
backend\pipeline\segmenters\fusion_hybrid.py:68:        return WasbRunner(wcfg),
{"weights_path": wcfg.weights_path, "fusion_substrate": "wasb"}
backend\pipeline\segmenters\fusion_hybrid.py:113:        (0) keep every window ? identical to
the substrate. Persisted candidates are
backend\eval\training.py:15:    persisted `stats`. Pluggable: a WASB substrate supplies its own
feature_fn.
backend\eval\training.py:19:substrate is chosen from the Phase 3 candidate-recall result.
backend\eval\training.py:72:    `feature_fn` is pluggable so a WASB substrate can supply
shuttle-motion
backend\pipeline\detectors\base.py:5:the high-accuracy shuttle trajectory substrate used by
tracknet_hybrid / wasb_hybrid.
tests\test_fusion_hybrid.py:5:substrate windows; served Gate A drops sub-threshold windows from
the AI handoff while
tests\test_fusion_hybrid.py:79:def
test_persisted_candidates_are_superset_of_substrate_windows():
tests\test_fusion_hybrid.py:82:    assert spans == [(1.0, 4.0), (10.0, 12.0)]          # exactly
the substrate windows, unfiltered
tests\test_fusion_hybrid.py:92:    # Persistence is still the full superset (offline substrate
intact)...
docs\ROADMAP.md:71:payoff ? a free, offline, self-improving rally mode. The **substrate was
chosen from
docs\ROADMAP.md:75:substrate is **WASB shuttle-motion** (ADR-001), not player-motion. See
**ADR-004**.
docs\ROADMAP.md:79:Once the substrate is WASB shuttle-motion, the next lever is **fusing
additional
docs\ROADMAP.md:114:| 4 | Offline learned segmenter (WASB shuttle-motion substrate) | ? Next
(substrate decided: WASB, ADR-004) |
docs\VIDEO_RECORDING_GUIDELINES.md:16:[Omitted long matching line]
```

8. assistant (2026-06-13T19:15:55.127Z)

Now let me verify the test_config.py claim about whether it checks routing or just config parsing.

9. user (2026-06-13T19:15:56.130Z)

```
25          assert cfg.indexing.ai_max_workers == 5
26
27
28      def test_fusion_config_defaults_reproduce_control():
```

```
29          """The fusion block defaults must be no-ops: wasb substrate, no cues, no gating, no
mask."""
30          cfg = load_config("/nonexistent/path/12345/config.json")
31          f = cfg.indexing.fusion
32          assert f.substrate == "wasb"
33          assert f.cue_flags == {}            # person cue OFF
34          assert f.min_crossings == 0         # Gate A OFF
35          assert f.min_players == 0           # Gate B OFF
36          assert f.court_polygon is None      # no mask
37          assert f.velocity_threshold == 0.02
38
39
40      def test_fusion_config_file_override():
41          with tempfile.TemporaryDirectory() as td:
42              p = os.path.join(td, "fusion.json")
43              with open(p, "w") as fh:
44                  json.dump({"indexing": {"fusion": {
45                      "substrate": "tracknet", "min_crossings": 2,
46                      "cue_flags": {"person": True}, "court_polygon": [[0.1, 0.1], [0.9, 0.1],
[0.5, 0.9]],
47                  }}}, fh)
48              cfg = load_config(p)
49              f = cfg.indexing.fusion
50              assert f.substrate == "tracknet"
51              assert f.min_crossings == 2
52              assert f.cue_flags == {"person": True}
53              assert len(f.court_polygon) == 3
54              # An old config without a fusion block still loads (defaults applied).
55              assert cfg.indexing.wasb.velocity_threshold == 0.02
56
57
58      def test_file_override():
59          with tempfile.TemporaryDirectory() as td:
```

10. user (2026-06-13T19:15:56.910Z)

```
docs\CODE_MAP.md:113:[Omitted long matching line]
backend\pipeline\segmenters\tracknet_hybrid.py:34:    def _build_runner(self, idx_cfg: Dict[str,
Any]) -> Tuple[DetectorRunner, Dict[str, Any]]:
backend\pipeline\segmenters\wasb_hybrid.py:36:    def _build_runner(self, idx_cfg: Dict[str,
Any]) -> Tuple[DetectorRunner, Dict[str, Any]]:
backend\pipeline\segmenters\fusion_hybrid.py:58:    def _build_runner(self, idx_cfg: Dict[str,
Any]) -> Tuple[DetectorRunner, Dict[str, Any]]:
backend\pipeline\segmenters\trajectory_hybrid.py:15:- ``_build_runner(idx_cfg)`` ? returns
(DetectorRunner, detector-specific
backend\pipeline\segmenters\trajectory_hybrid.py:53:    def _build_runner(self, idx_cfg:
Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str, Any]]:
backend\pipeline\segmenters\trajectory_hybrid.py:147:        runner, runner_params =
self._build_runner(idx_cfg)
tests\test_fusion_hybrid.py:47:    with patch.object(FusionSegmenter, "_build_runner",
return_value=(_runner(), {"weights_path": "w"})), \
tests\test_trajectory_hybrid_equivalence.py:48:    with patch.object(cls, "_build_runner",
return_value=(_runner(), {"weights_path": "w"})), \
```

11. assistant (2026-06-13T19:16:02.061Z)

Now let me verify whether the `TrackNetConfig.from_indexing_cfg` and `WasbConfig.from_indexing_cfg` accept `idx_cfg` and that the `queue_length` attribute exists, so that the suggested direct unit test would actually be feasible/correct.

12. user (2026-06-13T19:16:03.343Z)

```
backend\pipeline\detectors\tracknet_runner.py:37:    weights_path: str = ""
# WSL path to the .h5/.keras weights (REQUIRED)
```

```
backend\pipeline\detectors\tracknet_runner.py:38:    queue_length: int = 5
backend\pipeline\detectors\tracknet_runner.py:46:    def from_indexing_cfg(cls, idx_cfg:
Dict[str, Any]) -> "TrackNetConfig":
backend\pipeline\detectors\tracknet_runner.py:53:            weights_path=tn.get("weights_path",
""),
backend\pipeline\detectors\tracknet_runner.py:54:
queue_length=int(tn.get("queue_length", 5)),
backend\pipeline\detectors\tracknet_runner.py:130:        if not self.cfg.weights_path:
backend\pipeline\detectors\tracknet_runner.py:132:                "TrackNet weights_path is not
set. Set indexing.tracknet.weights_path "
backend\pipeline\detectors\tracknet_runner.py:184:            f"--model_weights
{wsl_tilde_quote(self.cfg.weights_path)} "
backend\pipeline\detectors\tracknet_runner.py:186:            f"--queue_length
{self.cfg.queue_length}"
```

13. user (2026-06-13T19:16:03.948Z)

```
backend\pipeline\detectors\wasb_runner.py:37:    weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
backend\pipeline\detectors\wasb_runner.py:45:    def from_indexing_cfg(cls, idx_cfg: Dict[str,
Any]) -> "WasbConfig":
backend\pipeline\detectors\wasb_runner.py:52:            weights_path=w.get("weights_path",
backend\pipeline\detectors\wasb_runner.py:149:            f"--weights
{wsl_tilde_quote(self.cfg.weights_path)} "
```

14. assistant (2026-06-13T19:16:22.015Z)

I now have all the facts needed to adjudicate. Let me verify each claim in the finding against the code.

Claim 1: `_build_runner` has a real branch on `fusion.substrate`. Confirmed ? `fusion_hybrid.py:58-68`. Line 59 reads `substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")`; line 60 branches; tracknet builds `TrackNetRunner(cfg)` with params `{weights_path, queue_length, fusion_substrate: "tracknet"}`; the default builds `WasbRunner(wcfg)` with `{weights_path, fusion_substrate: "wasb"}`.

Claim 2: Every fusion-hybrid test patches `_build_runner` out entirely. Confirmed ? `test_fusion_hybrid.py:47`: `patch.object(FusionSegmenter, "_build_runner", return_value=(_runner(), {"weights_path": "w"}))`. All five tests route through `_run()`, which uses this patch. So neither branch of the real `_build_runner` runs, and the returned `runner_params` (with `fusion_substrate` / `queue_length`) is never asserted. The mock return even omits `fusion_substrate`, confirming it isn't exercised.

Claim 3: `test_config` only checks parsing, not routing. Confirmed ? `test_config.py:40-55` (`test_fusion_config_file_override`) asserts `f.substrate == "tracknet"` on the parsed config object only; it never instantiates `FusionSegmenter` or calls `_build_runner`. So nothing asserts the segmenter routes on the substrate value.

Claim 4: The routing + returned params dict are pure and unit-testable directly. Confirmed feasible ? `_build_runner` only constructs config via `TrackNetConfig.from_indexing_cfg(idx_cfg)` / `WasbConfig.from_indexing_cfg(idx_cfg)` (both pure dict readers, `tracknet_runner.py:46`, `wasb_runner.py:45`) and constructs the runner. With `TrackNetRunner`/`WasbRunner` patched, calling `_build_runner` directly is pure logic. `cfg.queue_length` (`tracknet_runner.py:38`) and `cfg.weights_path` (`tracknet_runner.py:37`, `wasb_runner.py:37`) exist, so the suggested assertions are valid. The fix is accurate and achievable in this no-GPU environment.

Severity check: This is a genuine coverage gap ? the substrate-routing branch and the `fusion_substrate`/`queue_length` metadata tagging are entirely untested, and that metadata flows into `detection_params` persisted to the DB (`trajectory_hybrid.py:162`). But it's a low-severity test-coverage observation, not a correctness bug. The code itself is correct; only the test suite under-covers it. The finding self-rates "low" and explicitly frames it as a test gap, not a nitpick about style. That matches.

Everything in the finding holds against the actual code. No part is refuted.

15. user (2026-06-13T19:16:34.186Z)

Structured output provided successfully

16. assistant (2026-06-13T19:16:41.433Z)

I've verified the finding against the actual code and confirmed it holds (real=true, severity=low).

Summary of what I checked:
- `backend/pipeline/segmenters/fusion_hybrid.py:58-68` ? the `substrate` branch is real and returns distinct `runner_params` dicts (`fusion_substrate` tag, plus `queue_length` for tracknet).
- `tests/test_fusion_hybrid.py:47` ? all five tests route through one `_run()` helper that patches `_build_runner` out entirely, returning `{"weights_path": "w"}` (notably omitting `fusion_substrate`), so neither branch ever runs.
- `tests/test_config.py:40-55` ? only asserts the parsed config object (`f.substrate == "tracknet"`), never instantiates the segmenter or calls `_build_runner`, so routing is untested.
- `backend/pipeline/segmenters/trajectory_hybrid.py:162,216` ? confirmed `runner_params` flows into persisted `detection_params`, so the untested tag isn't inert.
- `tracknet_runner.py:37-38,46` and `wasb_runner.py:37,45` ? confirmed `from_indexing_cfg`, `queue_length`, and `weights_path` exist, so the suggested direct unit test is feasible and offline-runnable.

It's a genuine test-coverage gap (the substrate-routing branch and its metadata are entirely untested), correctly self-rated low since the production code itself is correct.